Report

Roll Number 2023101040

1. Implementation of Specifications:

Gotta count 'em all

proc.h:

Added a syscall tracking array (syscall_counts) for each process.

sysproc.c:

Implemented the getSysCount system call to retrieve the syscall count for a specific process.

Syscall Handler syscall.c:

Modified the syscall() function to increment syscall counts for every process upon each syscall invocation.

user/syscount.c:

Added the syscount user program to run a specified command and report how many times a particular syscall (identified by a mask) was invoked by the child process.

Wake me up when my timer ends

sysproc.c

sys_sigalarm: This system call sets an alarm for a process, specifying an interval (in ticks) and an alarm handler (a function that gets called when the alarm goes off). It initializes the process's alarm-related fields.

sys_sigreturn: This system call restores the process's state to what it was before the alarm handler was invoked, allowing the process to resume normal execution.

trap.c

Alarm Mechanism in usertrap: Every time a timer interrupt occurs, the process's tick count is incremented. If the tick count reaches the specified alarm interval and the alarm is not currently active, the process's state is saved, and the alarm handler is called. After the handler completes, sigreturn() can restore the saved state to resume the process from where it left off.

Lottery-Based Scheduling (LBS):

- **Changes made**:
    - Added a tickets field in proc.h to store the number of tickets each process holds.

  Implemented a system call settickets() to allow users to set the number of tickets for each process.
    - Implemented the lottery_pick() function to randomly select a process based on the proportion of tickets.
    - Updated the scheduler() function to use lottery_pick() when LBS is enabled.

Multi-Level Feedback Queue (MLFQ):

- **Changes made**:
    - Added new fields added to struct proc:
      int priority : This field stores the current priority level of the process in the Multi-Level Feedback Queue (MLFQ) scheduler.
      int time_slice :This field keeps track of the remaining CPU time slice that a process can use before it gets preempted
      int queue_time: This field tracks how long (in ticks) the process has been running in its current queue.
    - proc.c
      init_mlfq: Initializes the priority, time_slice, and queue_time fields for a process when it enters the MLFQ scheduler.

      priority is set to 0 (highest priority).
      time_slice is set to 1, meaning the process gets 1 time slice in this highest priority queue before being evaluated for demotion.
      queue_time is set to 0, indicating that the process has just entered the queue.
    - update_priority Function :

Instead of using a fixed 48 ticks for demotion, each priority level has a different time slice (time_slices[] array: 1, 2, 4, 8, 16, 32).
If a process spends more than the allowed time in its current queue (checked against queue_time), it is demoted to a lower-priority queue.

- boost_priority :
  Implements a gradual boost where processes are only boosted by one priority level at a time, rather than resetting all processes to the highest priority.

- **Queue-based Scheduling Logic:**
  The scheduler now prioritizes processes in lower-priority queues (higher priority number) when higher-priority queues are empty.
  For each runnable process in a queue:
  The process is set to RUNNING and context-switched using swtch.
  After the process runs, the queue_time is incremented.
  The process's priority is updated using the update_priority() function based on its time slice.

2. Performance Comparison between default (Round Robin), Lottery Based Scheduling, and Multi-Level Feedback Queue

Scheduler	Average Rtime	Average Wtime
Round Robin	11-12 ticks	110 ticks
MLFQ	12-15 ticks	144-159 ticks
LBS	14-15 ticks	103 ticks

Analysis:

RR provides consistent runtimes between 11-12 ticks. Since it is fair and non-prioritized, all processes receive roughly the same amount of CPU time.

MLFQ shows some variability in runtime (12-15 ticks). It is slightly higher than RR because the scheduler prioritizes interactive (I/O-bound) processes and demotes CPU-bound ones.

Wait times are higher, ranging between 144-159 ticks, which suggests CPU-bound processes in lower priority queues are waiting longer.

LBS has runtimes between 14-15 ticks, which is a bit higher than RR, suggesting some processes may be winning the lottery more often based on their ticket count.

LBS has the lowest wait time indicating that the randomness could reduce the waiting time for processes.

3. Implication of Adding Arrival Time to Lottery-Based Scheduling:

By incorporating the arrival time of processes into the lottery scheduling, we can prioritize older processes if multiple processes have the same number of tickets. This will ensure fairness: When two processes have the same ticket count, the older process (one that arrived earlier) will run first, preventing new processes from constantly getting scheduled ahead of older ones.

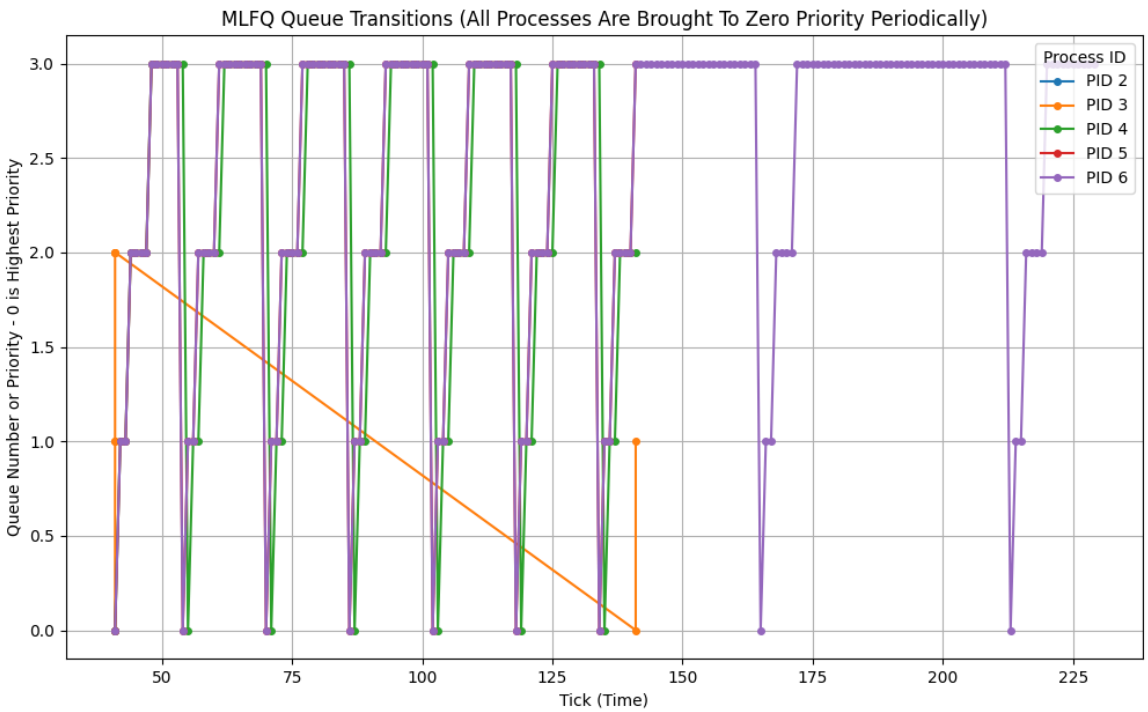
A disadvantage: It could be possible that while older processes with equal tickets are favored, newer processes may have to wait longer. For example, if we have 10 processes that all win 1 ticket, If a new process joins the system with the same 1 ticket, the scheduler might give preference to processes that arrived earlier (if tickets are equal), this new process might have to wait for all the older processes to get scheduled first.

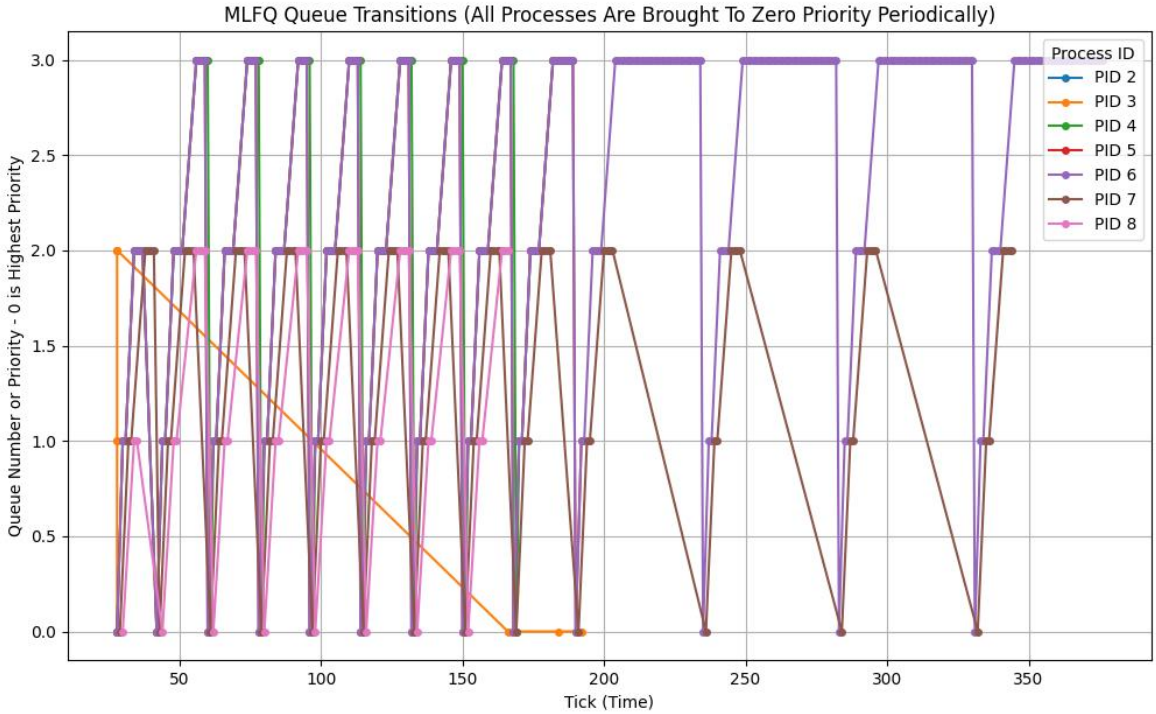
What happens if all processes have the same number of tickets?

Since all processes have the same number of tickets, they all have an equal chance of winning the lottery at each scheduling interval. LBS essentially turns into a random scheduler. (If arrival time is not considered).

4. MLFQ Graphs

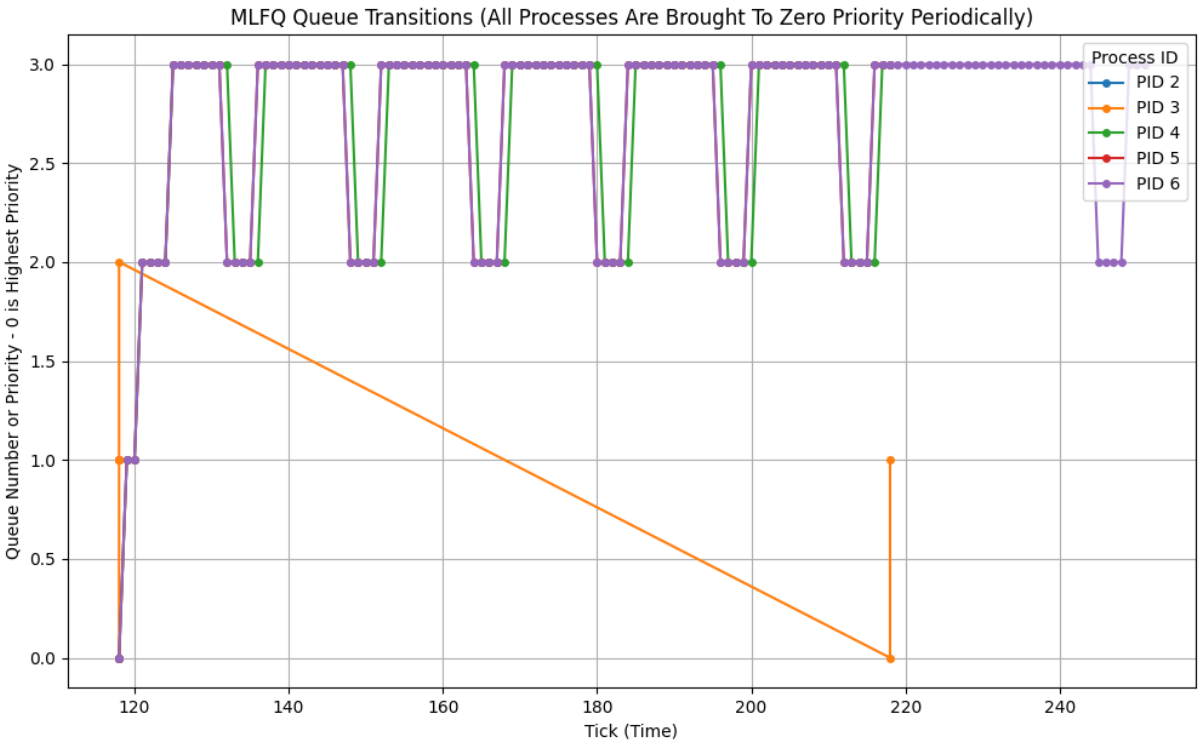
Both gradual and sudden boosts have been implemented.
Graph 1 and 2: All processes are periodically boosted to the highest priority queue (Queue 0) at fixed intervals. This creates a sharp, vertical reset in priority for every process. The sudden jump from lower queues to Queue 0 occurs at regular intervals, preventing starvation.

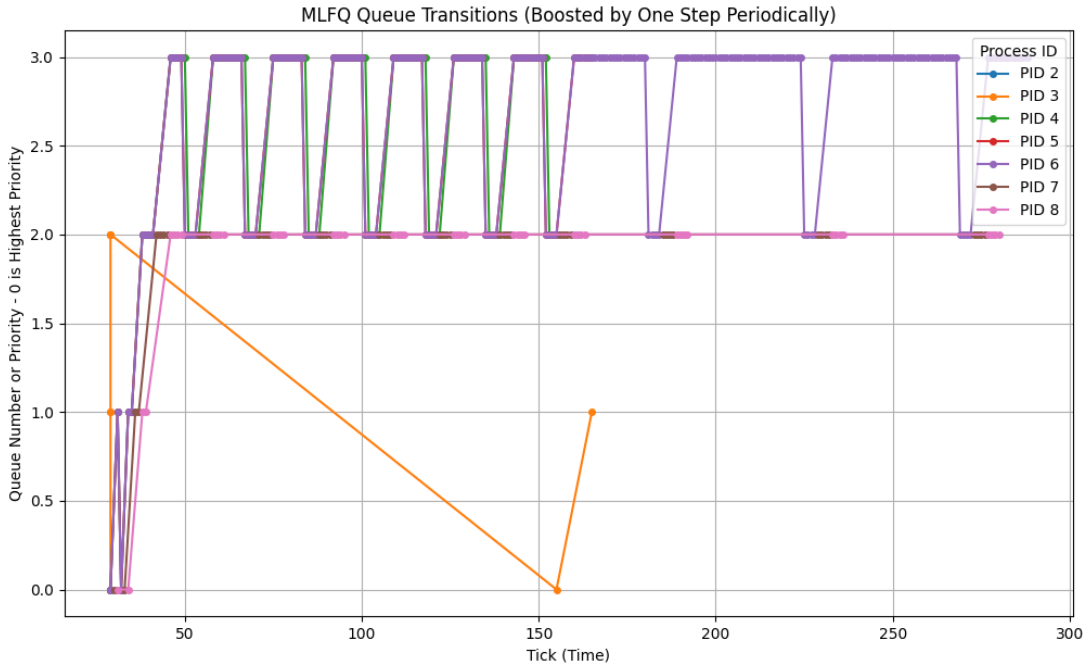




Graph 3 and 4: the boosting mechanism is more gradual, which means processes stay longer in lower queues, and the boost doesn't reset everyone at once.

The purple line for PID 6, for example, shows continuous cycling between queues, but it remains longer in the lower queues without immediate boosts.





Important commands:

For scheduler:

make clean; make qemu CPUS=1 SCHEDULER=MLFQ

make clean; make qemu CPUS=2 SCHEDULER=LBS

Networking:
./a.out 127.0.0.1 5000 5002

./a.out 127.0.0.1 5002 5000